

5.10

一、BERT 创新的具体思路

1. BERT 之前的相关工作

在 BERT 之前，并没有一个真正意义上的可以双向理解文本的模型。

- 浅层拼接方案：

2018年初提出的 ELMo 是 BERT 之前最著名的双向模型，其核心是训练两个独立的单向 LSTM（一个从左到右，一个从右到左），然后将产生的词向量拼接到一起。



这样的双向并不彻底，想象一句话：Attention is all you need.

从左到右训练时候，"is" 仍然只能关注到它左边的内容 "Attention"。

从右到左训练时候，"is" 只能关注到它右边的内容 "all"、"you"、"need"、"."。

将这两者拼接起来，似乎并不能做到真正的双向性。

而且没有采用 Transformer 架构。

- 单向微调方案：

OpenAI 于2018年6月发布 GPT，是单向模型。采用了更强大的 Transformer 架构，但是是一个自回归模型，采用了注意力掩码，某个 Token 只能关注到它左边的 Tokens。

2. BERT 的具体做法



如何做到真正的双向性？

Transformer 中的 Encoder 没有掩码机制，具有双向性，直接把原句子完整的输入进去，再设计一个简单的任务，让模型在这个过程中直接学习可以吗？

不行的，至少在普通的预测任务上是不行的，直接输入整个句子做注意力时候会让每个 Token 都关注到其他 Tokens，相当于直接看到答案了。



BERT 设计了两种巧妙的预训练任务，做到双向性。

词与词层面 MLM：随机抽取句子中的一些 Tokens，把他们变成 [MASK]，这是一种特殊 Token，将 Mask 之后的句子输入到 Encoder，再做预测。

句与句层面 NSP：即下一句预测，将样本分为正负样本。

正样本：B 是 A 的下一句话。

负样本：B 不是 A 的下一句话，是其他句子。

给模型一个样本，让模型判断这是正负样本中的哪个。（算一种二分类任务）。

3. 具体做法中更细节的部分

- MLM：随机抽取15%的 Tokens，80%变为 [MASK]，10%变为随机词，10%不变。
- 一些特殊 Token：
 - [CLS]：位置是每个输入序列的最开头，被看做融合了整个序列的信息，分类任务中只需要拿这个 Token 进行分类即可。
 - [SEP]：分隔两个句子。例：[CLS] 句子A [SEP] 句子B [SEP]。
 - [MASK]
 - [PAD]：统一句子长度，方便并行计算。

- 三个 Embedding：

分别告诉模型“这个词是什么”、“它属于哪句话”、“它在什么位置”。

- Token Embeddings: 对 Token 本身做 Embedding。
- Segment Embeddings: 每句话的所有 Tokens 共享同一个 Segment Embeddings。
- Position Embeddings: 包含位置信息，BERT 中采用可学习参数来生成。

最终输入 = Token Embeddings + Segment Embeddings + Position Embeddings。

二、对一些问题的理解

1. 计算 MLM 预训练损失时，参与计算的 Tokens 有哪些

参与损失计算的是被选中的 15% 的 Tokens，而不是其中被替换为 [MASK] 的那一部分，也不是所有 Tokens。

2. 在实现损失函数时，如何确保未被 Mask 的 Token 不参与损失计算

常用的实现方法是为损失函数设置忽略索引。在 PyTorch 中，`CrossEntropyLoss` 提供了 `ignore_index` 参数。我们只需将未被选中位置的标签设置为 `ignore_index`（例如 -100），而将被选中的 15% 位置的标签设为原始 Token 的 id。损失函数在计算时将自动跳过标签为 `ignore_index` 的位置，使它们的损失为 0，从而保证只有被 Mask 处理过的 Token 参与梯度更新。

3. BERT 的优缺点分别是什么

优点：

- 真正的双向上下文建模，在众多任务上取得最佳结果。
- 采用预训练 + 微调模式，不需要大改，只需少量任务数据即可适配下游任务。
- Transformer 层数更多。

缺点：

- 预训练计算开销大。
- MLM 引入 [MASK] 标记，但微调阶段并无该标记，造成预训练与微调之间的不一致。
- NSP 任务相对简单，部分后续工作证明其作用有限，甚至可被移除。
- 天然的自编码模型，不适合直接用于自回归文本生成，生成能力较弱。

4. BERT 怎么用在生成模型中

BERT 本身是自编码模型，不能直接像 GPT 那样进行自回归生成。但我猜想可以让其作为 Encoder。在 Encoder-Decoder 框架中，用 BERT 作为编码器，再随机初始化一个解码器训练。

三、理解代码 & 小型消融实验

1. Debug 代码

Debug 了代码，大致流程：

预训练：输入文本 -> 嵌入层 -> 通过掩码处理-> Transformer 编码器-> 输出向量 -> 通过 MLM 和 NSP 任务计算损失并更新参数。

微调：任务数据 -> 嵌入层 -> Transformer Encoder -> 池化 -> 任务特定层 -> 预测结果。



此处的池化不是 CNN 里面的池化，此处代码实现具体指的是：

将 [CLS] 提取出来做一次全连接，再经过一次 Tanh 激活。

数据集：WikiText-2

表1 结构参数

参数	值	说明
vocab_size	10,000	词表大小（含 5 个特殊
hidden_size	256	隐藏层维度 / 词嵌入
num_hidden_layers	4	Transformer 编码器层数（BEI
num_attention_heads	4	多头注意力头数（256/4=
intermediate_size	1,024	FFN 中间层维度（4×hid
max_position_embeddings	128	位置编码最大长/
max_seq_length	128	输入序列最大长/
hidden_dropout_prob	0.1	Dropout 概率
attention_probs_dropout_prob	0.1	注意力权重 Dropc

2. 消融对比

在小样本上进行了像原论文一样的消融实验，对比了预训练时候采用 MLM+NSP 与 MLM-only 的差异。

表2 控制变量

变量	MLM+NSP	MLM-only
数据集	20,640 句子对	20,640 句子
batch_size	32	32
epochs	15	15
nsp_labels	nsp_labels 传入	--no_nsp (nsp_label

```
nsp_loss = None
if nsp_labels is not None:
    # 二分类交叉熵损失
    loss_fn = nn.CrossEntropyLoss()
    nsp_loss = loss_fn(nsp_logits, nsp_labels)
```

图3.2.1 code 实现 MLM-only

总 Loss 是由 mlm_loss 与 nsp_loss 相加得到，在 MLM-only 时候直接不计算 nsp_loss 即可。

表3 实验数据

Epoch	MLM-only Loss	MLM-only PPL	MLM+NSP Total	MLM+NSP MLM部分	MLM+NSP PPL

1	8.09	3250	8.81	8.1	6714
3	7.01	1105	7.67	6.98	2148
5	6.84	937	7.54	6.85	1873
7	6.73	837	7.44	6.75	1701
10	6.61	745	7.3	6.61	1475
15	6.52	677	7.17	6.49	1297

3. 结论

- 1. 观察第二列与第五列，MLM 损失分量几乎相同，表明 NSP 不会对 MLM 产生损害。
- 2. 观察第一列与第三列，计算后得到 nsp_loss 约为0.69，这个值是均衡二分类随机猜测的理论值 (ln2)。这表明 NSP 任务没有学习到有效信号。



所以我们直接下结论，NSP 任务没有用...

是不行的，只能证明 NSP 任务在此模型规模下，此数据集下没有起到作用。

4. 数据

图例有点小瑕疵，没来得及更改。

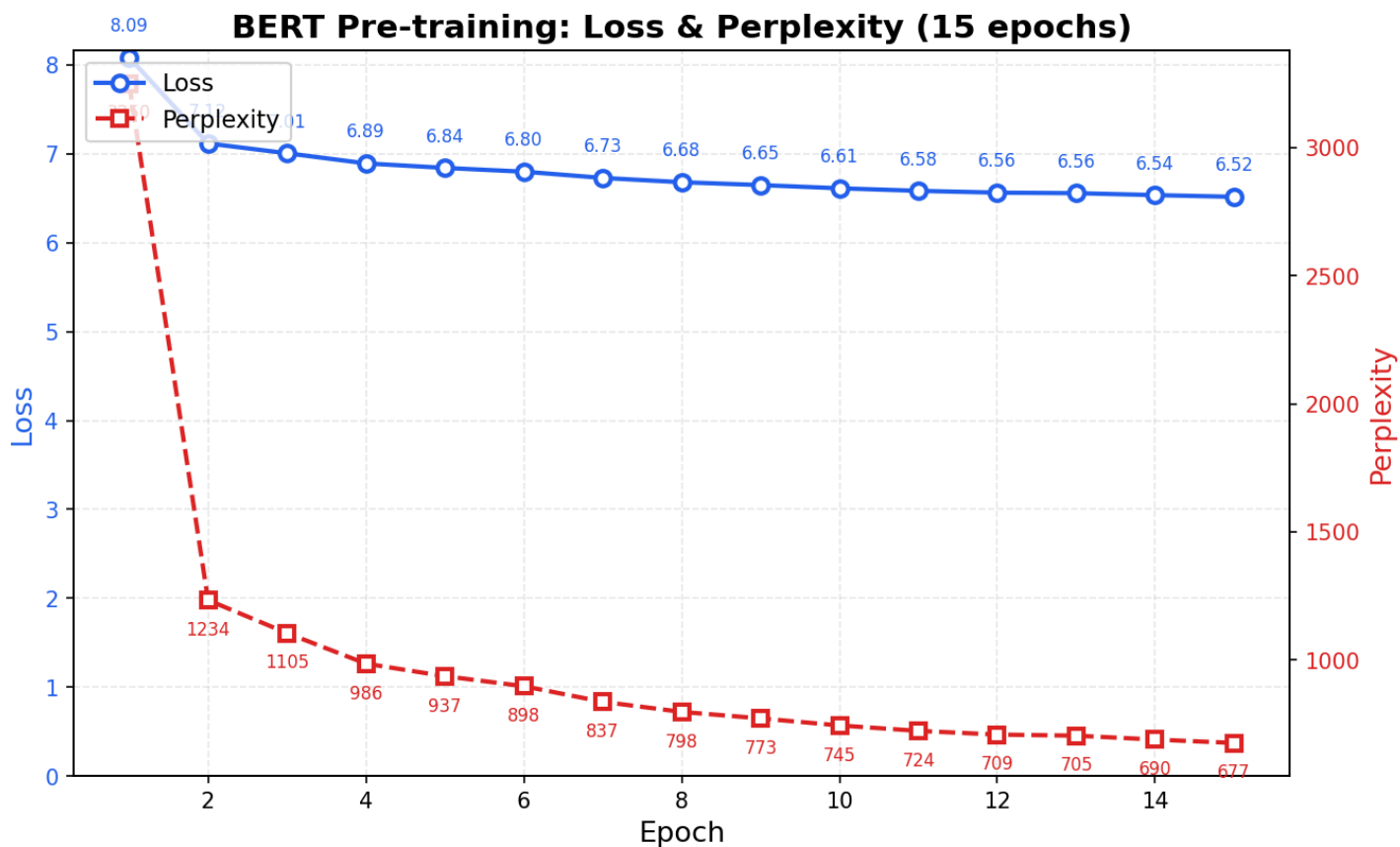


图3.4.1 MLM-only 预训练部分实验数据

$$\text{PPL} = e^{\text{Loss}}$$

图3.4.2 困惑度

困惑度是衡量模型预测能力的一个指标。

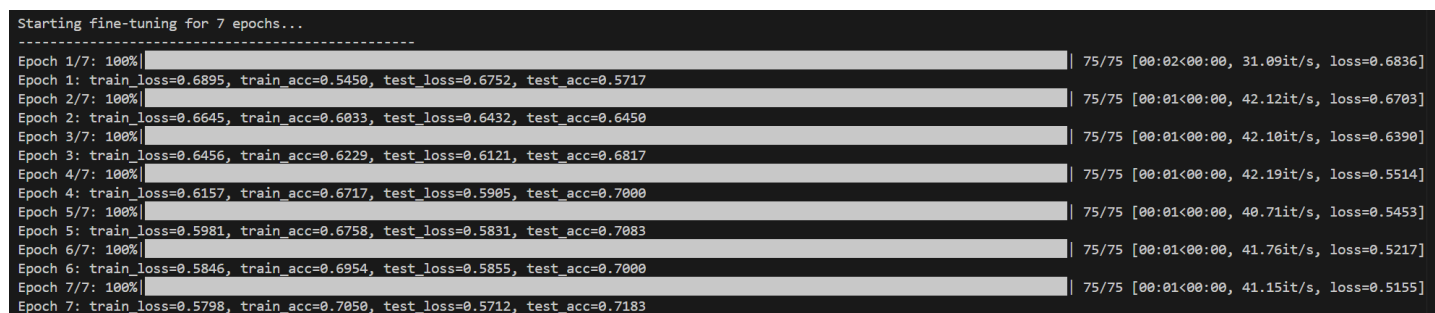


图3.4.3 微调数据